Appendix C Installation and User Guide

This guide installs ShopSteward from its repository on a Windows machine, starts it with the demonstration configuration, and walks through the four scenarios of Section 3.4. The code is in the SystemCode folder of the repository; commands are run in PowerShell from that folder unless stated otherwise. The same dependencies install on macOS and Linux; Section C.5 gives the per-process start commands for those systems.

C.1 Requirements

Table C-1. Software and accounts.

Item	Version or note
Git	any recent version
Python	3.12, managed with **uv**
Node.js	22, with **Corepack** (the repository pins pnpm 10.17.1)
Docker Desktop	running; it hosts the business PostgreSQL
OpenAI API key	for the language-model features; the demonstration configuration uses gpt-6.1-sol in every role
Free ports	3000 (web), 8000 (business API), 8001 (simulator), 55432 (PostgreSQL); 8053 if the forecast service is enabled

Model calls consume the API usage of the configured account. The routine restocking flow (S1 without the agent conversation) runs without any model.

C.2 Installation

```
git clone --branch submission https://github.com/yhnapoleon/ShopSteward.git
Set-Location ShopSteward/SystemCode
uv sync --frozen --all-packages --all-groups --python 3.12
corepack pnpm install --frozen-lockfile
```

`uv.lock` and `pnpm-lock.yaml` pin every Python and frontend dependency. To use the v6 forecast service, install the optional machine-learning extras as well (Section C.4).

C.3 Configuration and databases

Create the local settings files from their templates:

```
foreach ($m in 'infra', 'backend', 'simulation', 'frontend') {
    if (-not (Test-Path "$m/.env")) { Copy-Item "$m/.env.example" "$m/.env" }
}
```

Fill in the fields of Table C-2. Each credential is an independent random value; `[guid]::NewGuid().ToString('N')` produces one.

Table C-2. Required settings.

File	Fields
`infra/.env`	`POSTGRES_PASSWORD`; the port defaults to `POSTGRES_PORT=55432`

File	Fields
backend/.env	DATABASE_URL with the same password and port; AUTH_TOKENS with a local user (below); SIMULATION_TOKEN equal to the simulator's service token
simulation/.env	SIM_DATABASE_URL pointing to a separate shopsteward_sim database; SIM_SERVICE_TOKEN of at least 24 characters
frontend/.env	NUXT_BACKEND_URL=http://127.0.0.1:8000 (the Windows launcher sets the rest)

The business database URL has the form postgresql+asyncpg://shopsteward:<password>@127.0.0.1:55432/shopsteward. A local administrator is one line in backend/.env:

```
AUTH_TOKENS=[{"token":"<random user token, at least 24 characters>","principal_id":"local-admin","kind":"user","roles":["admin"],"store_ids":[]}]
```

Start PostgreSQL and create the simulator's database. Compose creates the business user and database; the simulator's role and database are created once by hand, with the password used in simulation/.env:

```
docker compose --env-file infra/.env -f infra/compose.yaml up -d --wait
docker compose --env-file infra/.env -f infra/compose.yaml exec postgres `
    psql -U shopsteward -d postgres
```

```
CREATE ROLE shopsteward_sim LOGIN;
\password shopsteward_sim
CREATE DATABASE shopsteward_sim OWNER shopsteward_sim;
\q
```

Apply the migrations from each module's directory, so that each reads its own .env:

```
Set-Location backend
..\.venv\Scripts\python.exe -m alembic upgrade head
Set-Location ../simulation
..\.venv\Scripts\python.exe -m alembic upgrade head
Set-Location ..
```

The business database should report 0020_business_time and the simulator sim_0003_controls (python -m alembic current).

C.4 Models and the forecast service

Demonstration configuration. infra/demo-mature.env.example holds the model settings used for the demonstration: gpt-6.1-sol through the Responses API for the conversational agent, the explanation step and the supplier reviewers, with the work-item and learning workers switched on. Copy it, for example to var/demo.env, and replace the two <path to the OpenAI key file> placeholders with the absolute path of a text file that contains only the API key. The file is laid over backend/.env at start-up (Section C.5), so backend/.env itself stays unchanged.

v6 forecast service (optional). The trained model bundle ships with the repository. Install the extras, check the bundle, and enable the service in backend/.env:

```
uv sync --frozen --all-packages --all-groups --all-extras --python 3.12
.venv/Scripts/python.exe ml/tools/reproduce_v6.py
```

```
FORECAST_V6_ENABLED=true
FORECAST_V6_URL=http://127.0.0.1:8053
FORECAST_V6_TOKEN=<random service token>
```

The check prints `verification: PASS`. Scenarios S1–S4 use the demand stated in each scenario and do not need the forecast service.

Knowledge service (optional). Document retrieval runs as a separate service with its own PostgreSQL and OpenSearch, deployed with `infra/compose.knowledge.yaml` and the knowledge runbook in `docs/runbooks/`. Scenarios S1–S4 do not use it.

C.5 Starting and stopping

Windows. One command starts the simulator, the business API, the business worker, the web frontend and, according to the settings, the agent, work-item and learning workers and the forecast service, and waits until each is ready:

```
.venv/Scripts/python.exe infra/start_windows.py --overlay var/demo.env
```

Open **http://127.0.0.1:3000**. The launcher writes its logs and the IDs of the processes it started to `var/windows-stack-<time>/processes.json`. To stop them:

```
$run = Get-ChildItem var -Directory -Filter 'windows-stack-*' |
    Sort-Object Name | Select-Object -Last 1
$record = Get-Content "$($run.FullName)/processes.json" -Raw | ConvertFrom-J
son
$record.processes | ForEach-Object { taskkill /PID $_.pid /T /F }
```

macOS and Linux, or one terminal per process. Activate the environment in each Python terminal (`source .venv/bin/activate`) and run:

Table C-3. Processes and their start commands.

Process	Directory	Command
Simulator	`simulation/`	`python -m simulator`
Business API	`backend/`	`python -m app`
Business worker	`backend/`	`python -m app.worker --profile business`
Agent worker	`backend/`	`python -m app.worker --profile agent`
Work-item worker	`backend/`	`python -m app.worker --profile work`
Learning worker	`backend/`	`python -m app.worker --profile learning`
Web frontend	SystemCode	`corepack pnpm --filter @shopsteward/frontend dev --host 127.0.0.1 --port 3000`

In this mode the model settings go into `backend/.env` directly, and `frontend/.env` also needs `NUXT_BACKEND_TOKEN` (the administrator token), `NUXT_DEV_TOOLS=true` and `NUXT_AGENT_ENABLED=true`.

C.6 Using the system

First steps.

1. Open http://127.0.0.1:3000. If the page asks for an identity, choose **Connect backend identity**.
2. Open **Settings → Language and region → Display language** and choose **English**.
3. In **Settings**, turn on **Demo mode**. It hides debugging labels and shows the presenter bar, whose **Next day** advances the simulated business by one scripted step and whose **New sample store** creates a fresh store.
4. Choose **Create sample store**. The sample store is a synthetic shop with 20 units of one product in stock, ¥1,000 in cash and an expected demand of 60 units.

The workspace. Today shows what needs a decision; **Follow-up** lists the delegated restocking tasks; **Activity** records business changes, the owner's decisions and the ledger; **Overview** charts cash and sales; **Document library** holds business documents. **Give me a task** hands the system a piece of work, such as processing a quotation, and the **Agent conversation** of a task answers questions and proposes changes. Every purchase waits for the owner's approval.

S1 Promotion restocking with follow-up.

1. On **Today**, open **Try a restocking comparison**, keep the cash floor at ¥300 and the quantities 0, 20, 40 and 80. The table recommends 40 units and marks 80 as leaving cash below the floor (Figure 3-4).
2. Choose **Start with these conditions** to turn the comparison into a follow-up task, open **Review 40-unit purchase**, check **View evidence**, and confirm the purchase of 40 units.
3. Press **Next day** on the presenter bar until the 40 units arrive, 10 units are sold and demand is revised from 60 to 70. A change card proposes 20 more units (Figure 3-5); approve it.
4. In the **Agent conversation**, ask the agent to explain the recommendation, then open **View context record** to see what entered the model's context.

S2 Supplier failure, multi-agent sourcing and the owner's order.

1. Create a second store with **New sample store** and open **Emergency restock**. Describe the shortage (about 10 units a day for a week) with a budget of ¥300.
2. Confirm and let the system find suppliers. One reviewer agent reads each sample supplier's documents; the comparison then recommends an option and shows each supplier's card with the quoted excerpts (Figure 3-6).
3. Adopt the recommended option and approve it.
4. Copy the generated order message. Paste a reply as the supplier would send it, stating the quantity, unit price, total and delivery date of that order; the system reads it into fields and shows the words each field came from (Figure 3-7).
5. Choose **Record arrival** to add the goods to stock.

S3 Quotation correction and reuse.

1. Open **Give me a task**, start a quotation task and attach sample quote A (¥120 per box of 12, minimum two boxes). The result shows ¥10 per unit and a minimum of 24 units.
2. Ask for quotations to be compared by unit price and choose to update and remember the sort order. The rule is saved as version 1.
3. Attach sample quote B (¥180 per box of 15, minimum three boxes). It is processed with the saved rule: ¥12 per unit and a minimum of 45 units (Figure 3-8).

S4 Memory, records and the business brief.

1. In the **Agent conversation**, write “Remember: show me cash and stock first.” The conversation confirms the update, and **What the system remembers** lists the preference with its version (Figure 3-9).
2. Open **Activity** to see the business changes, decisions and ledger of S1, and open the **Business brief** to read or download a summary of recent events, actions and next steps.

C.7 Troubleshooting

Table C-4. Common problems.

Symptom	Check
The API reports live but not ready	database connection; the migration head (`python -m alembic current` in `backend/`)
Setting up a store or a task stays queued	the business worker and the simulator are running, and the two service tokens match
The page shows no identity, or 401 errors	`AUTH_TOKENS` in `backend/.env`; reconnect the backend identity
The agent does not answer	`AGENT_ENABLED`, the key file path, the API mode and the agent worker
A quotation task is saved but never processed	`WORK_PROCESSOR_ENABLED` and the work-item worker
The forecast shows as unavailable	`FORECAST_V6_ENABLED`, the service on port 8053 and its token
The launcher reports a port in use	stop the earlier run (Section C.5) before starting again